Caller Registration Flow — Implementation Notes

Name → pincode → village capture in the Farm Vaidya voice agent (Bot.py / Agri Phero Solutions)

Project: pipecat · Prepared: 6 August 2026 · Scope: new-caller metadata capture and its supporting tools/data

1. What this flow does

Every new caller (one without a saved `dim_contacts` row for their phone number) goes through a short registration sequence before the bot answers substantive questions: it collects their **name**, resolves their **pincode** into a mandal/district/state, confirms their **village**, and saves all of it so the next call from that number skips this entirely. This same pincode-resolution machinery is reused a second time later in a call, when a caller asks for a market price or weather and needs their nearest Agmarknet market yard confirmed.

1. **Ask name** → caller answers → `save_caller_name(name)` is called immediately, before any spoken reply.
2. **Ask pincode** → caller dictates digits, often fragmented across turns (e.g. "5, 3, 3" then "4, 0, 1").
3. Each fragment is submitted to `collect_pincode_digits(digits)`, which deterministically accumulates them and reports back once exactly 6 digits are collected.
4. Once complete, `lookup_place_by_pincode(pincode)` resolves the pincode to candidate villages (1, several, or "too many to list") plus mandal/district/state.
5. Bot reads out candidates (or asks the caller to state their village directly, if too many), caller confirms/names their village.
6. `save_caller_location(...)` persists village + mandal + district + state + pincode to `dim_contacts`.

2. Components

2.1 Conversational rules — `system_prompt.txt`

The LLM's turn-by-turn behavior for this flow is entirely prompt-driven (no separate state machine in code). The relevant sections are **"New caller metadata capture"** and **"Caller location"**, which specify, in order:

- Ask name → call `save_caller_name` before any spoken reply to it.
- Ask pincode → submit every digit-bearing utterance to `collect_pincode_digits`, never count digits manually.
- On `lookup_place_by_pincode`'s result: read back 1 candidate for yes/no confirmation; read out 2–8 candidates as a list to pick from; for >8 candidates, ask for the village directly (no guessing, no yes/no) — see §4.1.
- Accept whatever village the caller states even if it isn't in the candidate list (hamlets under a shared pincode are common and not individually indexed).
- Call `save_caller_location` as the very next action once the village is settled.

2.2 Tools — `bot_processors/location_lookup.py`

Tool	Purpose	Registered in
`save_caller_name`	Persists the caller's name immediately on hearing it.	`Bot.py` tools list
`collect_pincode_digits` tool	Deterministic per-call digit accumulator. See §4.2 — this replaced an unreliable prompt-only approach.	`Bot.py` tools list
`lookup_place_by_pincode`	Pincode → candidate villages + mandal/district/state, from the AP post-office reference data.	`Bot.py` tools list
`confirm_location`	Pincode + spoken place + state → nearby Agmarknet market yards, ranked by distance. Used for price/weather lookups, not registration itself.	`Bot.py` tools list
`save_caller_location`	Persists village/mandal/district/state/pincode (and optionally market) to `dim_contacts`.	`Bot.py` tools list

2.3 Reference data

`lookup_place_by_pincode` reads `bot_processors/pincode_geo_reference.json` (built from `all_ap_postoffices.xlsx` via `pincode_geo_reference.py`) — currently Andhra Pradesh only. Only rows with `match_confidence == "matched"` are used (district cross-checked against Agmarknet's real district vocabulary), deduplicated by (pincode, post office).

1,159	470	27
distinct indexed pincodes	pincodes with >8 villages (40%)	max villages under one pincode (531118)

Full breakdown: `bot_processors/pincode_village_counts.xlsx` (per-pincode village lists + distribution summary).

2.4 Storage — `db/schema.sql`

```
CREATE TABLE IF NOT EXISTS dim_contacts (
    phone_number          TEXT PRIMARY KEY,
    name                  TEXT,
    first_seen            TIMESTAMPTZ NOT NULL,
    last_seen             TIMESTAMPTZ NOT NULL,
    call_count            INTEGER NOT NULL DEFAULT 0,
    confirmed_state       TEXT,
    confirmed_district    TEXT,
    confirmed_mandal      TEXT,
    confirmed_village     TEXT,
    confirmed_market      TEXT,
    confirmed_pincode     TEXT,
    location_confirmed_at TIMESTAMPTZ
);
```

Postgres (local instance, `farmvaidya` database, dedicated `farmvaidya_app` role scoped to just this database). On a later call, a system message states the caller's confirmed name/location so the entire registration sequence is skipped.

3. Full data flow

```
Caller                    Bot.py (LLM)              location_lookup.py tools
dim_contacts

"my name is X"     -----> save_caller_name(X) ----------------------------------->
name = X
"5, 3"              \
"3, 4, 0, 1"          --> collect_pincode_digits("53")   -> {complete:false,
needed:4}
                          collect_pincode_digits("3401") -> {complete:false,
needed:1}
                          ...until 6 digits reached...   -> {complete:true,
pincode:"533401"}
                      ---> lookup_place_by_pincode("533401")
                             -> 1 candidate:      {candidates:[v]}            ->
yes/no confirm
                             -> 2-8 candidates:   {candidates:[v1..v8]}       ->
read list, ask which
                             -> >8 candidates:    {too_many_to_list:true,
                                                    mandal, district, state,
                                                    all_villages:[...27]}     -> ask
for village directly

(all_villages only read

aloud if caller explicitly

asks to verify)
""    ---> save_caller_location(village, mandal,
                                district, state, pincode) ------------------------>
confirmed_* columns set
```

4. Issues found and fixed during implementation

These were found by tracing real call logs against the intended flow, not hypothetically — each entry below cites the actual log evidence.

4.1 Mandal name presented as a guessed village FIXED

> **Symptom:** for a >8-village pincode, the bot said "మీ ఊరు పుల్లంపేట మండలంలో ఉందని నేను అనుకుంటున్నాను. ఇది సరైనదేనా?" ("I think your village is in Pullampeta *mandal*. Is that right?") —

treating the mandal name as if it were a single guessed village and asking yes/no, when there is no single candidate to confirm.

Root cause: the `too_many_to_list` instruction said "don't read out a list" but never explicitly forbade guessing *one* village — the model found that gap.
Fix: `system_prompt.txt` now explicitly states the mandal is not the village, forbids presenting it as one, and forbids yes/no framing in this branch — the bot must go straight to an open "what's your village name?" question.

4.2 Unreliable digit concatenation FIXED

Symptom: callers had to repeat their pincode multiple times. One real call: caller said "53" then "1118" (exactly 6 digits total), and the bot still replied "దయచేసి మీ పిన్ కోడ్ ఆరు అంకెల్లో చెప్పగలరా?" ("please say your pincode in six digits") as if it hadn't received all 6.

Root cause: the original instruction asked the *LLM itself* to "silently concatenate every digit" across turns — precise counting/arithmetic over fragmented conversational turns is exactly the kind of bookkeeping LLMs are inconsistent at, especially with uneven-sized chunks (2 digits then 4, rather than neat groups of 3).
Fix: moved the counting into deterministic Python — the new `collect_pincode_digits` tool (`location_lookup.py`). The LLM's only job is to call it every time the caller says anything digit-like; the tool tracks a per-call buffer and reports `complete` / `digits_needed` itself. State lives in a plain closure variable (each call gets its own instance via `run_bot()`, which runs once per call), so there's no cross-call leakage and nothing to clean up.

```
async def collect_pincode_digits(params, digits: str = "", reset: bool = False) -> None:
    nonlocal _buffer
    new_digits = _clean_pincode(digits)
    _buffer = new_digits if reset else (_buffer + new_digits)
    if len(_buffer) >= 6:
        pincode, leftover = _buffer[:6], _buffer[6:]
        _buffer = ""
        result = {"complete": True, "pincode": pincode}
    else:
        result = {"complete": False, "digits_so_far": _buffer,
                  "digits_needed": 6 - len(_buffer)}
    await params.result_callback(result)
```

Verified against the exact failing case: `"53"` then `"1118"` → `{complete: true, pincode: "531118"}`, no repeats needed.

4.3 Crash on the self-correction path FIXED

Symptom: a live call crashed with `collect_pincode_digits() missing 1 required positional argument: 'digits'`. The caller had said "sorry, I gave the wrong pincode," and the bot tried `collect_pincode_digits(reset=True)` with no `digits` — exactly the correction pattern the tool's

> own docstring promised support for ("reset=true, digits can be empty"), but the parameter had no default value, so the call failed before entering the function body at all.

> **Fix:** `digits: str = ""`. Re-verified with the exact crash reproduction (reset called with no digits argument), a combined `reset=True, digits="507130"` case, and a regression check that normal accumulation still works — all pass. The call had actually self-recovered on the caller's next utterance even before this fix landed (the buffer was untouched by the crashed call), but the crash still produced a visible glitch mid-call and is now closed.

5. Design notes worth keeping in mind

- **The 8-candidate cutoff is deliberate, not arbitrary:** 470 of 1,159 pincodes have more than 8 villages (one has 27) — reading that many names aloud on a live call is unusable. Above that threshold the bot asks for the village as free text instead, since mandal/district/state are already known regardless of which exact village is named (they're geocoded per-taluk, identical for every village sharing a pincode).
- **Full village list is available but not spoken by default** even past the 8-candidate cutoff — `all_villages` is always included in `lookup_place_by_pincode`'s result, but the prompt only reads it aloud (in batches of ~8, with a check-back between batches) if the caller explicitly doubts their village is covered and asks to hear the names.
- **A named village that isn't on the list is accepted, not argued with** — hamlets under a shared pincode are common and not all individually indexed in the post-office data; the mandal/district/state from the lookup are still correct and are what pricing/weather actually key off, so the exact village name is mostly for personalization rather than lookup accuracy.
- **The same digit-accumulation problem existed in a second flow** (the price/weather "Caller location" section, via `confirm_location`) and was fixed identically — both flows now call `collect_pincode_digits` rather than asking the model to count digits itself.